

Plan de Refactorización Técnica e Incremental: `donaciones-service`

Propósito del documento: Este plan técnico establece la hoja de ruta ejecutable para refactorizar `donaciones-service`, corrigiendo los problemas de asignación de responsabilidades identificados en el diagnóstico, preservando el comportamiento existente y preparando el terreno para la futura persistencia relacional, sin incurrir en sobreingeniería ni reescrituras innecesarias.

0. Marco Teórico y Criterios de Ingeniería para el Proyecto

0.1 ¿Qué significa refactorizar en este contexto?

En el contexto de **DonaTrack** (proyecto multi-servicio con arquitectura por capas, DDD y desarrollo en equipo), refactorizar significa:

- **Preservación estricta del comportamiento observable:** El sistema debe responder exactamente los mismos códigos HTTP, payloads JSON y eventos asíncronos para las mismas entradas.
- **Reubicación de responsabilidades:** Mover lógica de negocio desde capas técnicas (por ejemplo, `infrastructure/` y listeners) hacia el Dominio o Aplicación según corresponda.
- **Aumento de cohesión y reducción de acoplamiento:** Dividir clases con múltiples motivos de cambio (God Services, switches de casos de uso disímiles) en componentes especializados.
- **Clarificación del modelo de dominio:** Hacer explícitas las invariantes y políticas de negocio, por ejemplo, extraer `PoliticaConsolidacionPropuestas` en lugar de esconderla en métodos privados estáticos.
- **Satisfacción de observaciones docentes sin reescritura:** Resolver los señalamientos de la cátedra con el menor número de cambios atómicos y verificables.

0.2 ¿Qué NO debería considerarse refactor?

- **No reescribir microservicios o módulos:** Cambiar la arquitectura completa a hexagonal pura o reactiva solo por estética.
- **No incorporar nuevos features o reglas de negocio no solicitadas.**

- **No introducir prematuramente JPA/Hibernate:** Agregar anotaciones `@Entity`, `@Table` o repositorios de base de datos relacional cuando la entrega actual exige almacenamiento en memoria.
- **No reemplazar frameworks o librerías:** Cambiar Spring Web por WebFlux, o Spring Events por otra tecnología sin justificación funcional.
- **No realizar optimización prematura de rendimiento:** Alterar estructuras de datos o algoritmos que ya cumplen los tiempos de respuesta requeridos.
- **No realizar renombramientos masivos o cosméticos** que no cambien responsabilidades y solo generen conflictos de merge en Git.

0.3 Refactors necesarios vs. peligrosos/innecesarios antes de la entrega

Refactors necesarios	Motivo	Refactors peligrosos o innecesarios	Motivo
Mover Dominio fuera de Infrastructure (REF-03, REF-04)	Observación crítica de arquitectura	Desacoplar Agregados Necesidad ↔ DI por IDs (P8)	Rompe cálculo en memoria y algoritmos
Separar Listener de Caso de Uso (REF-05)	El listener no debe ser dueño del CU	Introducir capas de Ports/Adapters para Feign clients	Agrega capas vacías sin polimorfismo
Modularizar switches monolíticos (REF-07)	Evita baja cohesión y bugs futuros	Migrar repositorios en memoria a JPA antes de tiempo	Incompatible con alcance de Entrega 1
Eliminar fugas HTTP en Application (REF-01)	Limpieza de capas y uniformidad	Reemplazar Spring Events por brokers externos internos	Riesgo de integración innecesario

0.4 Code Smells frecuentes en un equipo de 10 integrantes

En proyectos grupales con división de roles, suelen emerger problemas característicos:

1. **Anemic Domain Model (Modelo Anémico):** Entidades convertidas en meras bolsas de getters/setters (`@Data` indiscriminado), mientras los `@Service` ejecutan todas las validaciones e invariantes de negocio.
2. **Duplicated Traversal (Navegación Duplicada):** Diferentes integrantes escriben independientemente la misma cadena de consultas para resolver un dato (por ejemplo, `DonacionIndependiente → Donacion → Donante → Persona`).
3. **God Service / Swiss-Army Service:** Servicios que acumulan 10 o más dependencias inyectadas y métodos para cualquier operación relacionada con una palabra clave (por ejemplo,

`DonacionesIndependientesService` agrupando 6 flujos operativos).

4. **Feature Envy (Envidia de Atributos):** Un servicio le pide repetidamente datos a una entidad para calcular algo que la entidad o un Value Object podrían calcular por sí mismos (`Tell`, `Don't Ask`).
5. **Divergent Representation (Modelos Divergentes):** Cada microservicio modela una versión distinta del mismo concepto (por ejemplo, `Persona` o `Donante`) con atributos incompatibles sin un contrato claro de replicación o sincronización.
6. **Leaky Framework Concerns (Fuga de Framework):** Clases de dominio importando librerías de infraestructura (por ejemplo, `@JsonIgnore` de Jackson en entidades o `ResponseStatusException` de Spring en servicios).

0.5 Detección de responsabilidades mezcladas entre servicios

Para identificar si dos servicios tienen responsabilidades solapadas o límites difusos:

- **El síntoma del "Query-then-Mutate":** Si el Servicio A le pide toda la información interna al Servicio B por REST/Feign para tomar una decisión de negocio y luego le manda la orden modificada al Servicio B, la lógica de decisión debería pertenecer directamente al Servicio B.
- **Doble fuente de verdad:** Si dos servicios mutan y persisten el mismo dato conceptual, por ejemplo, el estado de una entrega siendo administrado en paralelo por donaciones y logística sin un evento de orquestación claro.
- **Contratos hinchados:** DTOs de eventos que envían grafos enteros de objetos en lugar de identificadores y datos mínimos de cambio de estado.

0.6 Criterio para separar Dominio, Aplicación e Infraestructura

Capa	Responsabilidades
DOMAIN	Invariantes de negocio, entidades ricas, Value Objects, State Pattern, Domain Services, algoritmos de matching, políticas, puertos, interfaces y contratos de repositorios. No debe importar Spring Web, Jackson, Feign ni SQL.
APPLICATION	Casos de uso, coordinación y secuencia de operaciones, transacciones, invocación de repositorios y eventos, Mappers Entity ↔ DTO, helpers de resolución entre agregados y fachadas de orquestación frente a Presentation.
INFRASTRUCTURE	Adaptadores técnicos, listeners RabbitMQ y Spring Events, clientes HTTP/Feign, implementaciones de repositorios, integraciones externas, analizadores y normalizadores semánticos.

0.7 Preparación del código para Persistencia Relacional (JPA/Hibernate)

Para que el paso a base de datos relacional en la siguiente entrega sea fluido y sin fricción:

- **Identificadores Inmutables (Surrogate Keys):** Todas las entidades tienen un `UUID id` único e inmutable, preparado para claves primarias sin depender de datos naturales que puedan ser anonimizados, según DTI-01.
 - **Encapsulamiento de Colecciones:** Las entidades exponen listas mediante `Collections.unmodifiableList()`, obligando a usar métodos de negocio (`agregarItem()`, `quitarItem()`) en lugar de mutar la lista directamente.
 - **Value Objects Inmutables:** Clases como `Bien`, `BienNormalizado` e `ItemDonacion` implementadas como Java `record`, listas para mapearse como `@Embeddable` en JPA.
 - **Postergación de P8:** Mantener temporalmente `List<DonacionIndependiente>` dentro de `NecesidadExtraordinaria` y reemplazarlo por colección de IDs o tabla intermedia `@OneToMany` únicamente al redactar los mapeos ORM de JPA.
-

0.8 Preparación para Integración Asincrónica Robusta

- **Idempotencia en Listeners:** Asegurar que si un evento (`DonacionNormalizadaEvent` o `EventoRutaIniciada`) se entrega más de una vez, bajo semántica *at-least-once*, la aplicación no duplique registros ni corrompa estados.
 - **Eventos Inmutables y Livianos:** Los eventos de dominio transportan identificadores y valores mínimos necesarios (`UUID donacionId`, `UUID necesidadId`), delegando a la capa de aplicación la carga de entidades.
 - **Aislamiento del Listener:** El `@EventListener` o `@RabbitListener` solo desempaqueta el payload y delega a un Application Service con manejo de excepciones y logging adecuado.
-

0.9 Protocolo de Validación de No-Regresión

Para certificar que ningún refactor rompa el código previo:

1. **Línea base (Baseline):** Ejecutar `mvn clean test` en todo el repositorio antes de comenzar la Fase 1 y guardar el reporte de tests exitosos.
2. **Tests de Caracterización:** Proteger los endpoints existentes con `@WebMvcTest` y tests de integración (`CrossServiceCommunicationIT.java`).
3. **Validación PR por PR:** Ninguna PR se mergea si no pasan el 100 % de los tests unitarios y de integración de forma local y en el pipeline de GitHub Actions.

4. **Verificación en Preproducción:** Ejecutar `./run-preprod-tests.sh` con Docker Compose para validar la interacción entre los 5 microservicios levantados.
-

1. Validación del Diagnóstico Previo

A continuación se resume la validación técnica de cada hallazgo contrastado directamente contra el código fuente:

- **P1(`ProcesadorDeDonaciones` en `infrastructure/`) – CONFIRMADO:** Es un Application Service que orquesta el caso de uso asíncrono de normalización de donación, persiste entidades y dispara eventos de aplicación (`DonacionNormalizadaEvent`), pero reside erróneamente en la capa de infraestructura.
- **P2(`SegmentacionEventListener` con orquestación completa) – CONFIRMADO:** Aloja el caso de uso completo de segmentación, guardado de donaciones independientes, persistencia y comunicación Feign con incentivos dentro del listener. Debe separarse el Adaptador de Eventos (técnico) del Application Service.
- **P3(Algoritmos de asignación en `infrastructure/algoritmos/`) – CONFIRMADO:** `AlgoritmoAsignacion` , sus variantes y `StockDeDonaciones` contienen lógica de dominio pura y deben residir en la capa de Dominio (`models/domainServices/algoritmos/`).
- **P4(`DonacionesIndependientesService.cambiarEstado()` monolítico) – CONFIRMADO:** Un único método con `switch` maneja 6 transiciones de negocio heterogéneas con efectos colaterales disímiles. Debe modularizarse por comportamiento semántico.
- **P5(`AlgoritmosService` con baja cohesión) – CONFIRMADO:** Mezcla ejecución de algoritmos, políticas de consolidación, ciclo de vida de propuestas y notificaciones. Debe descomponerse claramente frente a `PropuestaService` .
- **P6(`Necesidad.toDTO()` en el Dominio) – CONFIRMADO:** La entidad de dominio conoce la clase `NecesidadDTO` . Debe extraerse a un Mapper formal.
- **P8(Referencia directa `NecesidadExtraordinaria` → `List<DonacionIndependiente>`) – POSPUESTO:** Se mantiene en memoria para esta iteración. Su desacoplamiento a nivel de IDs se pospone para la fase de persistencia JPA/relacional (DTI-01), evitando romper `cantidadAcumulada()` y algoritmos prematuramente.
- **P9(`ResponseStatusException` en Application) – CONFIRMADO:** Fuga de preocupación HTTP en capa de aplicación. Debe reemplazarse por `RecursoNoEncontradoException` / `ValidationException` .
- **P10(`@JsonIgnore` en `DonacionIndependiente`) – CONFIRMADO:** Dependencia de Jackson en el dominio; debe eliminarse.

- **P12 (Traversal repetido de actores) – CONFIRMADO:** Navegación relacional
`DonacionIndependiente` → `Donacion` → `Donante` → `Persona` repetida en 3 clases.
 Se extraerá a un `DonacionActorResolver`.
- **P14 (`consolidar()` privado estático en servicio) – CONFIRMADO:** Regla de dominio oculta en un método privado. Se extraerá a `PoliticaConsolidacionPropuestas`.

2. Arquitectura Objetivo de `donaciones-service`

2.1 Capa de Presentación

Componente	Responsabilidad
<code>DonacionesController</code>	<code>POST /api/donaciones</code> , <code>GET</code>
<code>DonacionesIndependientesController</code>	<code>PATCH .../estado</code>
<code>AsignacionController</code>	<code>POST /api/asignaciones/ejecutar</code>
<code>PropuestasController</code>	<code>GET</code> , <code>PUT .../estado</code>
<code>NecesidadesController</code>	Operaciones relacionadas con necesidades

2.2 Capa de Aplicación

Componente	Responsabilidad
<code>DonacionesService</code>	Carga y consulta de donaciones
<code>NormalizacionDonacionesService</code>	Orquesta normalización asíncrona
<code>SegmentacionDonacionesService</code>	Orquesta segmentación e incentivos
<code>AsignacionDonacionesService</code>	Orquesta ejecución de algoritmos
<code>PropuestaService</code>	Gestión de propuestas y ciclo de vida
<code>DonacionesIndependientesService</code>	Orquesta transiciones operativas
<code>DonacionActorResolver</code>	Resolución de Donante y Persona
<code>mappers/</code>	Mappers Entity ↔ DTO

2.3 Capa de Dominio

Componente	Responsabilidad
------------	-----------------

Componente	Responsabilidad
models/entities/donaciones/	Donacion , Bien , Item
models/entities/donacionesIndependientes/	Entidades y State Pattern
models/entities/necesidades/	Necesidad y entidades relacionadas
models/entities/propuestas/	Propuesta y eventos
models/domainServices/algoritmos/	Algoritmos y servicios de dominio
AlgoritmoAsignacion	Template para algoritmos de matching
AlgoritmoCompatibilidadSemantica	Compatibilidad semántica
AlgoritmoPrioridadSubAtendidos	Priorización de subatendidos
StockDeDonaciones	Gestión de stock
PoliticaConsolidacionPropuestas	Política de consolidación
models/ports/	Puertos, como Segmentador
models/repositories/	Interfaces de repositorios

2.4 Capa de Infraestructura

Componente	Responsabilidad
infrastructure/events/	Listeners y adaptadores de eventos
clients/	Clientes HTTP y Feign
analizadores/	Integraciones de análisis
segmentadores/	Implementaciones técnicas de segmentación
LogisticaListener	Integración con eventos de logística
repositories/impl/	Implementaciones concretas de repositorios

3. Mapa de Responsabilidades: Antes → Después

Clase	Responsabilidad Actual	Problema	Responsabilidad Objetivo	Acción
ProcesadorDeDonaciones	Normalización async en infrastructure/	Ubicación incorrecta	Application Service: normalización	Renombrar y mover a services/impl/NormalizacionDonacionesService

Clase	Responsabilidad Actual	Problema	Responsabilidad Objetivo	Acción
<code>SegmentacionEventListener</code>	Segmentación, incentivos y persistencia	Caso de uso en listener	Adaptador de eventos	Conservar como listener técnico y delegar a <code>SegmentacionDonacionesService</code>
Nueva: <code>SegmentacionDonacionesService</code>	No existía	Caso de uso sin servicio	Application Service: segmentar e incentivos	Crear en <code>services/impl/</code>
<code>AlgoritmoAsignacion</code> y subclases	Matching en <code>infrastructure/</code>	Dominio en infraestructura	Domain Services / Domain Algorithms	Mover a <code>models/domainServices/algoritmos/</code>
<code>StockDeDonaciones</code>	Gestión de stock en <code>infrastructure/</code>	Dominio en infraestructura	Domain Helper	Mover a <code>models/domainServices/algoritmos/</code>
Nueva: <code>PoliticaConsolidacionPropuestas</code>	Método privado estático	Regla de negocio oculta	Domain Policy	Extraer a <code>models/domainServices/</code>
<code>AlgoritmosService</code>	Algoritmos + Propuestas + Notificaciones	Baja cohesión	Application Service: ejecución de matching	Renombrar a <code>AsignacionDonacionesService</code> y delegar propuestas a <code>PropuestaService</code>
<code>PropuestaService</code>	Wrapper parcial + Event listener	Responsabilidad dividida	Application Service: ciclo de vida de propuestas	Asumir gestión de estado, historial y aprobación
<code>DonacionesIndependientesService</code>	Switch monolítico en <code>cambiarEstado()</code>	Baja cohesión	Casos de uso semánticos + dispatcher	Exponer métodos semánticos
<code>Necesidad</code>	Contiene <code>toDTO()</code> en la entidad	Acoplamiento Dominio → DTO	Entidad pura de dominio	Remover <code>toDTO()</code> y crear <code>NecesidadMapper</code>
Nueva: <code>NecesidadMapper</code>	No existía	Mapeo en entidad	Mapper formal DTO ↔ Entity	Crear en <code>services/mappers/</code>
<code>DonacionIndependiente</code>	<code>@JsonIgnore</code> en <code>asignadaA</code>	Acoplamiento con Jackson	Entidad pura de dominio	Remover anotación <code>@JsonIgnore</code>

Clase	Responsabilidad Actual	Problema	Responsabilidad Objetivo	Acción
Nueva: <code>DonacionAct orResolver</code>	Traversal duplicado en 3 clases	Duplicación de consultas	Componente de aplicación	Crear en <code>services/impl/</code> o <code>services/</code>

4. Detalle de los Refactors Propuestos

REF-01: Corrección de Fuga de Excepciones HTTP en Application Service

- **Objetivo:** Eliminar `ResponseStatusException` de `AlgoritmosService` y utilizar las excepciones estándar de `common-lib`.
- **Problema actual:** `AlgoritmosService` lanza `ResponseStatusException(HttpStatus.NOT_FOUND)` y `BAD_REQUEST`, acoplando la capa de aplicación con el framework Web/HTTP.
- **Evidencia:** `AlgoritmosService.java`, líneas 174 y 179.
- **Responsabilidad actual:** Application Service maneja códigos de estado HTTP directamente.
- **Responsabilidad objetivo:** Application Service lanza excepciones de dominio o aplicación (`RecursoNoEncontradoException`, `ValidationException`).
- **Clases afectadas:** `grupo5.donaciones.services.impl.AlgoritmosService`.
- **Métodos afectados:** `actualizarEstadoPropuesta(UUID id, EstadoPropuesta estado)`.
- **Cambios conceptuales:** Desacoplar la capa de aplicación del protocolo HTTP.
- **Cambios estructurales:** Reemplazar `new ResponseStatusException(HttpStatus.NOT_FOUND)` por `new RecursoNoEncontradoException(id)` y `BAD_REQUEST` por `new ValidationException(ErrorCatalog.ARGUMENTO_INVALIDO)`.
- **Dependencias / Precondiciones:** `GlobalExceptionHandler` de `common-lib` ya disponible.
- **Riesgos:** Ninguno.
- **Tests:** `PropuestasControllerTest.java`, `AlgoritmosServiceTest.java`.
- **Criterio de aceptación:** Cero imports de `ResponseStatusException` en `services/`.
- **Complejidad:** Muy baja.
- **Riesgo:** Bajo.

REF-02: Desacoplamiento de DTOs y Jackson del Dominio

- **Objetivo:** Remover `Necesidad.toDTO()` y `@JsonIgnore` de `DonacionIndependiente`, delegando la transformación a un Mapper formal.
- **Problema actual:** `Necesidad` conoce y referencia `NecesidadDTO`.
`DonacionIndependiente` tiene anotaciones de serialización web (`@JsonIgnore`).
- **Evidencia:** `Necesidad.java`, líneas 49 a 62; `DonacionIndependiente.java`, línea 30.
- **Responsabilidad actual:** Las entidades conocen la representación externa y detalles de serialización.
- **Responsabilidad objetivo:** Dominio puro sin dependencias de DTOs ni librerías de serialización.
- **Clases afectadas:** `Necesidad.java`, `NecesidadExtraordinaria.java`, `NecesidadRecurrente.java`, `DonacionIndependiente.java`, `NecesidadesService.java`, nueva clase `NecesidadMapper.java`.
- **Métodos afectados:** `Necesidad.toDTO()`, `NecesidadesService.guardar()`, `NecesidadesService.obtenerPorId()`, `NecesidadesService.listarConFiltros()`.
- **Cambios conceptuales:** Separación estricta entre modelo de dominio y contrato de transferencia.
- **Cambios estructurales:**
 1. Crear `grupo5.donaciones.services.mappers.NecesidadMapper` como `@Component`.
 2. Eliminar métodos `toDTO()` en `Necesidad` y subclases.
 3. Eliminar `@JsonIgnore` en `DonacionIndependiente`.
 4. Inyectar `NecesidadMapper` en `NecesidadesService`.
- **Riesgos:** Errores de mapeo en tests que consumían `toDTO()`.
- **Tests:** `NecesidadesServiceTest.java`, `NecesidadesControllerTest.java`, nuevo `NecesidadMapperTest.java`.
- **Criterio de aceptación:** El paquete `models/entities/` no tiene dependencias hacia `grupo5.donaciones.dto` ni `com.fasterxml.jackson`.
- **Complejidad:** Baja.

- **Riesgo:** Bajo.
-

REF-03: Reubicación de Algoritmos de Asignación y Política de Consolidación al Dominio

- **Objetivo:** Mover los algoritmos de asignación y stock desde `infrastructure/algoritmos/` hacia `models/domainServices/algoritmos/`, y extraer la política de consolidación de propuestas con visibilidad pública de instancia.
- **Problema actual:** Lógica de asignación de dominio pura reside en `infrastructure/`. La regla de consolidación está oculta en un método privado estático de `AlgoritmosService`.
- **Evidencia:** `infrastructure/algoritmos/AlgoritmoAsignacion.java`; `AlgoritmosService.java`, líneas 71 a 96.
- **Responsabilidad actual:** Infraestructura aloja algoritmos de negocio; regla de consolidación atrapada en servicio.
- **Responsabilidad objetivo:** El Dominio contiene las políticas de asignación, algoritmos y reglas de consolidación. `PoliticaConsolidacionPropuestas` es una clase de dominio instanciable con método público para permitir testing unitario aislado con fixtures complejas.
- **Clases afectadas:**
 - Mover `AlgoritmoAsignacion`, `AlgoritmoCompatibilidadSemantica`, `AlgoritmoPrioridadSubAtendidos` y `StockDeDonaciones` a `grupo5.donaciones.models.domainServices.algoritmos`.
 - Nueva clase `PoliticaConsolidacionPropuestas` en `grupo5.donaciones.models.domainServices`.
 - `AlgoritmosService.java` para actualizar imports y delegar consolidación.
- **Métodos afectados:** `AlgoritmosService.consolidar()`, `AlgoritmosService.ejecutar()`.
- **Cambios conceptuales:** Reclasificación de Domain Services y Domain Policies según DDD.
- **Cambios estructurales:** Repackaging de 4 clases de algoritmos, creación de `PoliticaConsolidacionPropuestas` y actualización de imports.
- **Riesgos:** Ajuste de imports en tests de algoritmos.

- **Tests:** `AlgoritmoCompatibilidadSemanticaTest` , `AlgoritmoPrioridadSubAtendidosTest` , `StockDeDonacionesTest` , nuevo `PoliticaConsolidacionPropuestasTest` .
 - **Criterio de aceptación:** El directorio `infrastructure/algoritmos/` deja de existir; `PoliticaConsolidacionPropuestas` se prueba unitariamente de forma aislada.
 - **Complejidad:** Baja.
 - **Riesgo:** Bajo.
-

REF-04: Reubicación de Normalización de Donaciones a Application Service

- **Objetivo:** Transformar `ProcesadorDeDonaciones` en un Application Service formal (`NormalizacionDonacionesService`) en el paquete `services/impl/` .
- **Problema actual:** `ProcesadorDeDonaciones` está en `infrastructure/` pero orquesta el caso de uso de normalización asíncrona, persiste entidades y avanza estados.
- **Evidencia:** `infrastructure/ProcesadorDeDonaciones.java` .
- **Responsabilidad actual:** Infraestructura orquesta el flujo de carga y normalización.
- **Responsabilidad objetivo:** Application Service orquesta el caso de uso `normalizarDonacionAsync(Donacion)` coordinando analizadores y repositorios.
- **Clases afectadas:**
 - `infrastructure/ProcesadorDeDonaciones.java` → mover y renombrar a `services/impl/NormalizacionDonacionesService.java` , implementando `INormalizacionDonacionesService` .
 - `DonacionesService.java` para inyectar la nueva interfaz.
- **Métodos afectados:** `DonacionesService.cargarDonacion()` , `NormalizacionDonacionesService.procesar()` .
- **Cambios conceptuales:** Claridad de que el procesamiento de normalización es un caso de uso de la aplicación.
- **Cambios estructurales:** Creación de interface en `services/` , implementación en `services/impl/` , eliminación en `infrastructure/` . Spring `@Async` se mantiene en el nuevo servicio.

- **Riesgos:** Modificación de inyección en `DonacionesService` .
 - **Tests:** Renombrar y adaptar `NormalizacionDonacionesServiceTest.java` , `DonacionesServiceTest.java` .
 - **Criterio de aceptación:** `infrastructure/` no contiene servicios de orquestación de normalización.
 - **Complejidad:** Baja.
 - **Riesgo:** Bajo.
-

REF-05: Desacoplamiento de Segmentación: Listener Fino + Application Service

- **Objetivo:** Separar la responsabilidad de Adaptador de Eventos (técnica) del Caso de Uso de Segmentación (aplicación).
- **Problema actual:** `SegmentacionEventListener` en `infrastructure/events/` contiene toda la lógica de segmentación, coordinación con incentivos y mutación de estado.
- **Evidencia:** `SegmentacionEventListener.java` , líneas 41 a 88.
- **Responsabilidad actual:** El listener técnico es dueño del caso de uso completo y posee 9 dependencias.
- **Responsabilidad objetivo:**
 - `SegmentacionEventListener` (Infrastructure): captura `DonacionNormalizadaEvent` y delega inmediatamente a la aplicación con 1 dependencia.
 - `SegmentacionDonacionesService` (Application): ejecuta el caso de uso `segmentarDonacion(UUID donacionId)` .
- **Clases afectadas:**
 - `infrastructure/events/SegmentacionEventListener.java` como adaptador fino.
 - Nueva interfaz `grupo5.donaciones.services.ISegmentacionDonacionesService` .
 - Nueva implementación `grupo5.donaciones.services.impl.SegmentacionDonacionesService` .

- **Métodos afectados:** `SegmentacionEventListener.onDonacionNormalizada()` .
 - **Cambios estructurales:** Extraer la lógica de orquestación del listener hacia el nuevo servicio.
 - **Riesgos:** Ninguno funcional si se preserva la firma del evento.
 - **Tests:** `SegmentacionDonacionesServiceTest.java` como test unitario puro y `SegmentacionEventListenerTest` para verificar delegación.
 - **Criterio de aceptación:** `SegmentacionEventListener` tiene solo 1 dependencia inyectada y no contiene lógica de negocio ni persistencia directa.
 - **Complejidad:** Media.
 - **Riesgo:** Bajo.
-

REF-06: Extracción de `DonacionActorResolver` para Notificaciones

- **Objetivo:** Unificar la lógica repetida de navegación entre agregados (`DonacionIndependiente` → `Donacion` → `Donante` → `Persona`) en un servicio o helper reutilizable de la capa de aplicación con nomenclatura clara.
- **Problema actual:** Código duplicado de traversal en `DonacionesIndependientesService` , `SegmentacionEventListener` y `AlgoritmosService` .
- **Evidencia:** Métodos privados equivalentes en 3 servicios distintos.
- **Responsabilidad actual:** Cada servicio navega manualmente múltiples repositorios para armar DTOs de notificación e incentivos.
- **Responsabilidad objetivo:** `DonacionActorResolver` en `services/` o `services/impl/` resuelve las identidades requeridas a partir de una `DonacionIndependiente` o `Donacion` .
- **Clases afectadas:**
 - Nueva clase `grupo5.donaciones.services.impl.DonacionActorResolver` .
 - `DonacionesIndependientesService.java` .
 - `SegmentacionDonacionesService.java` .
 - `AlgoritmosService.java` o `PropuestaService.java` .
- **Métodos afectados:** `obtenerDonanteId()` , `obtenerPersonaDonanteId()` , `obtenerPersonaBeneficiariaId()` .

- **Cambios estructurales:** Creación de la clase `DonacionActorResolver` e inyección en los servicios consumidores.
 - **Riesgos:** Muy bajo.
 - **Tests:** Nuevo `DonacionActorResolverTest.java`.
 - **Criterio de aceptación:** No existen métodos privados duplicados para navegación `Donacion` → `Donante` → `Persona` en los servicios.
 - **Complejidad:** Baja.
 - **Riesgo:** Muy bajo.
-

REF-07: Modularización por Comportamiento de `DonacionesIndependientesService` y Desacoplamiento de `LogisticaEventListener`

- **Objetivo:** Dividir el método monolítico `cambiarEstado()` en métodos de caso de uso semánticos y cohesivos, manteniendo el dispatcher para retrocompatibilidad con el Controller, y actualizar `LogisticaEventListener` para que invoque directamente los métodos semánticos.
- **Problema actual:** Un único método `cambiarEstado()` con `switch` ejecuta 6 transiciones radicalmente diferentes. Además, `LogisticaEventListener` construye DTOs artificiales para invocar el dispatcher genérico.
- **Evidencia:** `DonacionesIndependientesService.java` , líneas 66 a 90;
`LogisticaEventListener.java` , líneas 36 a 92.
- **Responsabilidad actual:** Servicio monolítico y listener técnico acoplado a un DTO genérico de cambio de estado.
- **Responsabilidad objetivo:**
 - `DonacionesIndependientesService` expone métodos específicos y semánticos por caso de uso:
 - `asignar(UUID id, UUID necesidadId, String actor)`
 - `iniciarTraslado(UUID id, String urlMapa, String actor)`
 - `confirmarEntrega(UUID id, String patenteCamion, String actor)`

- registrarFallaEntrega(UUID id, String justificacion, Boolean replanificable, String actor)
 - marcarVencida(UUID id, String actor)
 - retornarAlDeposito(UUID id, String actor)
 - cambiarEstado(UUID, DTO, actor) como dispatcher para el Controller REST.
 - LogisticaEventListener invoca directamente iniciarTraslado() , confirmarEntrega() , etc., sin instanciar CambioEstadoDonacionIndependienteRequestDTO .
 - **Clases afectadas:** IDonacionesIndependientesService.java , DonacionesIndependientesService.java , LogisticaEventListener.java .
 - **Riesgos:** Bajo. Los flujos internos ya están casi aislados en métodos privados; se formalizan como métodos de caso de uso.
 - **Tests:** DonacionesIndependientesServiceTest.java , LogisticaEventListenerTest.java .
 - **Criterio de aceptación:** LogisticaEventListener llama directamente a métodos semánticos; el dispatcher enruta limpiamente; tests unitarios cubren cada método por separado.
 - **Complejidad:** Media.
 - **Riesgo:** Bajo.
-

REF-08: Reestructuración y Alta Cohesión entre AsignacionDonacionesService y PropuestaService

- **Objetivo:** Clarificar las responsabilidades: AsignacionDonacionesService ejecuta el matching de donaciones y PropuestaService gestiona el ciclo de vida, confirmación y aprobación de propuestas.
- **Problema actual:** AlgoritmosService ejecuta algoritmos pero también aprueba propuestas, maneja eventos y notifica. PropuestaService actúa en parte como wrapper y en parte como event listener.
- **Evidencia:** AlgoritmosService.java y PropuestaService.java .
- **Responsabilidad objetivo:**

- `AsignacionDonacionesService` (Application): orquesta la ejecución de algoritmos sobre necesidades y donaciones, aplica `PoliticaConsolidacionPropuestas` y guarda las propuestas generadas.
- `PropuestaService` (Application): gestiona el listado, consulta, cambio de estado, aprobación (`confirmar`), fragmentación de donaciones y despacho a logística/notificaciones.

- **Clases afectadas:**

- Renombrar `AlgoritmosService` → `AsignacionDonacionesService` , implementando `IAsignacionDonacionesService` .
- Mover `aprobarPropuesta()` , `actualizarEstadoPropuesta()` y `listarPropuestas()` íntegramente a `PropuestaService` .
- `AsignacionController.java` y `PropuestasController.java` para inyectar interfaces.

- **Riesgos:** Ajuste de inyección de dependencias en controllers.

- **Tests:** `AsignacionControllerTest` , `PropuestasControllerTest` , `AsignacionDonacionesServiceTest` , `PropuestaServiceTest` .

- **Criterio de aceptación:** `AsignacionDonacionesService` tiene solo 4 dependencias; `PropuestaService` gestiona el ciclo de vida completo de propuestas; controllers inyectan interfaces.

- **Complejidad:** Media.

- **Riesgo:** Medio.

REF-09: Creación del Diagrama PlantUML de Dominio y Capas de `donaciones-service`

- **Objetivo:** Documentar el modelo de dominio y casos de uso en `docs/design/donaciones-service/diagrama-de-clases-donaciones.puml` , reflejando fielmente el diseño resultante.
- **Problema actual:** Falta documentación gráfica de clases para el servicio principal del proyecto.
- **Responsabilidad objetivo:** Diagrama PlantUML claro, estructurado con la paleta de estilo común (`donatrack-style.puml`), que muestre:

- Entidades de dominio con su comportamiento(`Donacion` , `DonacionIndependiente` , `Necesidad` , `Propuesta` , `Bien`).
 - El State Pattern de `DonacionIndependiente` .
 - Los Domain Services, algoritmos de asignación y políticas de consolidación.
 - Los puertos(`Segmentador`)y contratos de repositorio.
 - La relación conceptual con los casos de uso principales sin ruido accidental.
- **Clases afectadas:** Nuevo archivo `docs/design/donaciones-service/diagrama-de-clases-donaciones.puml` .
 - **Riesgos:** Ninguno sobre el código.
 - **Criterio de aceptación:** El archivo `.puml` compila limpiamente y el dominio permite comprender el negocio aun si se ocultan los servicios de aplicación.
 - **Complejidad:** Media.
 - **Riesgo:** Nulo.
-

5. Secuencia y Dependencias entre Pull Requests

PR 1: Limpieza de Dominio y Desacoplamiento Básico

Refactor	Acción
REF-01	Resolver fuga HTTP en Application
REF-02	Sacar <code>toDTO</code> y <code>@JsonIgnore</code> del Dominio y crear <code>NecesidadMapper</code>
REF-03	Mover algoritmos y stock al Dominio y extraer <code>PoliticaConsolidacion</code>

Resultado esperado: Dominio limpio y testeable de forma aislada.

↓

PR 2: Pipeline de Carga, Normalización y Segmentación

Refactor	Acción
REF-06	Crear <code>DonacionActorResolver</code> en Application
REF-04	Crear <code>NormalizacionDonacionesService</code>

Refactor	Acción
REF-05	Crear <code>SegmentacionDonacionesService</code> y convertir el listener en un adaptador fino

Resultado esperado: Pipeline asíncrono ordenado y sin duplicación.



PR 3: Modularización Operativa de Donaciones Independientes

Refactor	Acción
REF-07	Crear métodos semánticos y desacoplar <code>LogisticaEventListener</code>

Resultado esperado: Eliminación del switch monolítico.



PR 4: Cohesión de Asignación y Ciclo de Vida de Propuestas

Refactor	Acción
REF-08	Separar <code>AsignacionDonacionesService</code> y <code>PropuestaService</code>

Resultado esperado: Separación clara entre Matching y Lifecycle.



PR 5: Documentación Arquitectónica Oficial

Refactor	Acción
REF-09	Crear diagrama PlantUML oficial de <code>donaciones-service</code>

Resultado esperado: Documentación arquitectónica completa y alineada con el diseño final.

6. Qué NO Refactorizar: Protección de Diseño Existente

1. **Controllers de todos los microservicios:** Cumplen su rol como adaptadores HTTP limpios, validando DTOs y delegando a la aplicación.

2. **State Pattern de `DonacionIndependiente`** : Las clases `EnDeposito` , `AsignacionRealizada` , `ListaParaEntregar` , `EnTraslado` , `Entregada` , `EntregaFallida` , `Vencida` implementan un patrón ejemplar con invariantes protegidas.
 3. **Entidades con transiciones directas (`Donacion` , `Propuesta`)**: Sus métodos (`marcarNormalizada()` , `marcarSegmentada()` , `confirmar()`) encapsulan correctamente sus estados.
 4. **Logística-service en su totalidad**: Sus entidades (`Entrega` , `Ruta` , `Camion` , `Chofer`), servicios y publicación de eventos RabbitMQ están excelentemente desacoplados.
 5. **Notificaciones-service e Incentivos-service**: No requieren cambios de estructura interna.
 6. **Mecanismo de comunicación asíncrona (RabbitMQ y Spring Events)**: Las anotaciones `@RabbitListener` , `@EventListener` y `@Async` se conservan como herramientas técnicas de infraestructura.
 7. **common-lib** : Su jerarquía de excepciones (`BusinessStateException` , `ValidationException` , etc.) y repositorios base (`CrudRepository`) no se modifican.
 8. **Referencia directa `NecesidadExtraordinaria` → `DonacionIndependiente` (P8)**: Se pospone expresamente para la etapa de base de datos relacional (DTI-01), evitando sobreingeniería prematura.
 9. **ADRs existentes**: Se mantienen como histórico inmutable de decisiones.
-

7. Estrategia de Testing y Validación

7.1 Dominio Puro

- Verificar transiciones invariantes de `DonacionIndependiente` y sus clases de estado.
- Verificar `PoliticaConsolidacionPropuestasTest` de forma aislada con fixtures complejas de propuestas superpuestas y disjuntas.
- Verificar que los algoritmos de asignación ordenen y filtren idénticamente en su nueva ubicación.

7.2 Servicios de Aplicación

- `NormalizacionDonacionesServiceTest` y `SegmentacionDonacionesServiceTest` validando el encadenamiento de guardado y disparo de eventos.
- `DonacionActorResolverTest` validando resolución de IDs de personas donantes y beneficiarias.
- `DonacionesIndependientesServiceTest` testeando cada método semántico (`iniciarTraslado` , `confirmarEntrega` , etc.) por separado.

7.3 Controladores y Adaptadores

- `@WebMvcTest` asegurando que ningún cambio de servicio altere códigos HTTP ni contratos JSON.
- `LogisticaEventListenerTest` verificando que los eventos de RabbitMQ invocan directamente los métodos semánticos.

7.4 Pruebas de Integración E2E

- Ejecutar la suite `CrossServiceCommunicationIT.java` para validar el flujo completo inter-servicio:

`Donaciones` → `Incentivos` → `Notificaciones` → `Logística`

8. Resumen Ejecutivo de Refactors

Refactor	Prioridad	Beneficio	Complejidad	Riesgo	Pull Request
REF-01 (HTTP Exceptions)	Alta	Elimina fuga de presentación en Application	Muy baja	Muy bajo	PR 1
REF-02 (DTOs/Jackson fuera de Dominio)	Alta	Dominio puro sin acoplamiento externo	Baja	Bajo	PR 1
REF-03 (Algoritmos/Stock al Dominio)	Alta	Corrige ubicación de lógica pura de negocio	Baja	Bajo	PR 1
REF-06 (<code>DonacionActorResolver</code>)	Media	Elimina código duplicado de navegación	Baja	Muy bajo	PR 2
REF-04 (Normalización a App Service)	Crítica	Corrige caso de uso ubicado en infraestructura	Baja	Bajo	PR 2
REF-05 (Separar Listener de Segmentación)	Crítica	Convierte listener en adaptador fino	Media	Bajo	PR 2
REF-07 (Modularizar DonacionesIndependientes)	Alta	Elimina método monolítico y llamadas directas en listener	Media	Bajo	PR 3

Refactor	Prioridad	Beneficio	Complejidad	Riesgo	Pull Request
REF-08 (Separar Asignación y Propuestas)	Alta	Cohesión en servicios y controladores	Media	Medio	PR 4
REF-09 (Diagrama PlantUML)	Alta	Documentación arquitectónica completa	Media	Nulo	PR 5
P8 (Referencia agregados Necesidad ↔ DI)	Media	Desacoplamiento estricto de IDs	Alta	Alto	POSPONER (Persistencia)

9. Resultado Esperado

Al finalizar los refactors propuestos, `donaciones-service` deberá conservar su comportamiento observable mientras presenta una separación de responsabilidades más clara:

Aspecto	Estado objetivo
Dominio	Libre de dependencias HTTP, DTO y Jackson
Aplicación	Responsable de coordinar casos de uso
Infraestructura	Responsable de adaptadores e integraciones técnicas
Algoritmos	Ubicados en el Dominio
Listeners	Delgados y orientados a delegación
Servicios	Separados por responsabilidad y caso de uso
Propuestas	Gestionadas por un servicio especializado
Donaciones independientes	Transiciones modeladas mediante operaciones semánticas
Testing	Cobertura por capa y validación de no-regresión
Persistencia	Preparada para futura migración a JPA sin anticiparla
Documentación	Diagrama PlantUML alineado con la arquitectura final

Conclusión: La estrategia prioriza cambios pequeños, verificables y reversibles. El objetivo no es rediseñar `donaciones-service` , sino corregir las responsabilidades que actualmente están ubicadas en capas incorrectas, reducir duplicación y preparar el código para las siguientes etapas del proyecto sin introducir complejidad innecesaria.